



SpeechToTextWidget Documentation

A reusable Flutter widget that converts **speech to text** with a glowing microphone animation. It uses the `speech_to_text` package for voice recognition and `avatar_glow` for visual feedback while listening.

This version also exposes a **listening state callback**, allowing parent widgets to react when speech recording starts or stops.

Packages Used

```
dependencies:  
  flutter:  
    sdk: flutter  
  speech_to_text: ^6.6.0  
  avatar_glow: ^3.0.1  
  flutter_screenutil: ^5.9.0
```

Features

-  Tap-to-start / tap-to-stop voice recording
-  Animated glowing mic while listening
- Real-time speech-to-text conversion
-  Callback with recognized text
- Listening state callback (`onListening`)
-  Accepts initial text
- Responsive sizing with `ScreenUtil`

How It Works

1. Initializes speech recognition on widget load
2. Starts listening when microphone is tapped
3. Emits `true` via `onListening` when recording starts
4. Updates text live as the user speaks
5. Sends recognized text back via `onResult`
6. Emits `false` via `onListening` when recording stops
7. Stops listening automatically on dispose

Widget Parameters

Parameter	Type	Description
<code>onResult</code>	<code>Function(String)</code>	Returns recognized speech text
<code>onListening</code>	<code>Function(bool)?</code>	Notifies listening state (true/false)
<code>initialText</code>	<code>String?</code>	Pre-filled text (optional)

Example Usage

```
String _speechText = '';
bool _isListening = false;

SpeechToTextWidget(
  initialText: "Say something...",
  onResult: (text) {
    print("Recognized: $text");
  },
  onListening: (isListening) {
    debugPrint(isListening ? "Listening started" : "Listening stopped");
  },
)
```

Full Source Code

```
import 'package:avatar_glow/avatar_glow.dart';
import 'package:flutter/material.dart';
import 'package:flutter_screenutil/flutter_screenutil.dart';
import 'package:isaacmwesigw352/assets_helper/app_colors.dart';
import 'package:speech_to_text/speech_to_text.dart';

class SpeechToTextWidget extends StatefulWidget {
  final Function(String text) onResult;
  final Function(bool isListening)? onListening;
  final String? initialText;

  const SpeechToTextWidget({
    super.key,
    required this.onResult,
    this.onListening,
  })
```

```

        this.initialText,
    });

    @override
    State<SpeechToTextWidget> createState() => _SpeechToTextWidgetState();
}

class _SpeechToTextWidgetState extends State<SpeechToTextWidget> {
    final SpeechToText _speech = SpeechToText();
    bool _isListening = false;
    String _text = '';

    @override
    void initState() {
        super.initState();
        _text = widget.initialText ?? '';
        _initSpeech();
    }

    Future<void> _initSpeech() async {
        await _speech.initialize();
    }

    void _toggleListening() {
        if (_isListening) {
            _speech.stop();
            setState(() => _isListening = false);
            widget.onListening?.call(false);
        } else {
            _speech.listen(
                listenMode: ListenMode.dictation,
                onResult: (result) {
                    setState(() {
                        _text = result.recognizedWords;
                    });
                    widget.onResult(_text);
                },
            );
            setState(() => _isListening = true);
            widget.onListening?.call(true);
        }
    }

    @override
    void dispose() {
        _speech.stop();
        super.dispose();
    }
}

```

```

@override
Widget build(BuildContext context) {
  return Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
      Text(
        _text.isEmpty ? '' : _text,
        style: const TextStyle(fontSize: 16),
      ),
      const SizedBox(height: 20),
      AvatarGlow(
        glowShape: BoxShape.circle,
        animate: _isListening,
        glowColor: AppColors.c0F86A4,
        curve: Curves.easeInCirc,
        duration: const Duration(milliseconds: 1000),
        repeat: true,
        child: SizedBox(
          width: 96.w,
          height: 96.w,
          child: Material(
            shape: const CircleBorder(),
            child: Ink(
              decoration: BoxDecoration(
                gradient: LinearGradient(
                  colors: [
                    AppColors.c4F46E5.withAlpha(100),
                    AppColors.c3B82F6.withAlpha(100),
                  ],
                  begin: Alignment.topCenter,
                  end: Alignment.bottomCenter,
                ),
              shape: BoxShape.circle,
            ),
            child: InkWell(
              customBorder: const CircleBorder(),
              onTap: _toggleListening,
              child: Center(
                child: Icon(
                  _isListening ? Icons.mic : Icons.mic_none,
                  size: 28.sp,
                  color: Colors.white,
                ),
              ),
            ),
          ),
        ),
      ),
    ],
  ),
)

```

```
        ),  
    ),  
    ],  
    );  
}  
}
```

Notes

- Add microphone permission in **AndroidManifest.xml** and **Info.plist**
- Call `ScreenUtilInit()` in your app root
- `onListening` is optional but useful for disabling buttons or showing loaders

Common Use Cases

- Voice search with external UI feedback
- Chat input with typing lock while listening
- Accessibility features
- Voice-driven forms

Happy Coding 🎨